Java Programming — Class 9

Computation as Message Passing

Understanding Objects, Methods & How They Talk to Each Other

In real life, when you want something done, you **ask** someone to do it. You don't do everything yourself! The same idea applies in Java. A program is made up of **objects**, and these objects communicate by sending **messages** (calling methods) to each other. This is the heart of **Object-Oriented Programming (OOP)**.

1. What is an Object?

An **object** is a thing in your program — just like things in the real world. Every object has two parts:

- **State** (attributes / fields) — what the object *knows* about itself.
- **Behaviour** (methods) — what the object *can do*.

Real-World Object	State (what it knows)	Behaviour (what it can do)
Dog	name, breed, age	bark(), eat(), sleep()
Car	colour, speed, fuel	accelerate(), brake(), honk()
Student	name, marks, grade	study(), attendClass(), getMarks()
Phone	model, battery%, contacts	call(), sendSMS(), playMusic()

2. What is a Method?

A **method** is a block of code inside a class that performs a specific task. Think of a method as a *skill* that an object has. When you want the object to use that skill, you **call** the method — this is exactly like sending a message to the object.

■ **Tip:** Method = Action the object can perform. Calling a method = Sending a message to the object.

Basic Structure of a Method in Java:

```
returnType  methodName( parameters ) {
    // instructions go here
}

// Example:
void bark() {
    System.out.println("Woof! Woof!");
}
```

3. Sending a Message (Calling a Method)

To make an object do something, you use the **dot (.) operator** followed by the method name. This is called **method invocation** — or in OOP terms, **sending a message**.

Java Syntax	What it means in plain English
`dog.bark();`	Hey dog, please bark!
`car.accelerate(50);`	Hey car, accelerate to 50 km/h!
`student.getMarks();`	Hey student, what are your marks?
`account.deposit(500);`	Hey account, deposit ■500!

4. Examples of Message Passing

■ Example — 1 — Dog Speaks

We create a **Dog** class with fields (name, breed) and methods (bark, eat). Then we create a dog *object* and send it messages.

```
// ■■ Class Definition ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class Dog {
    // State (fields)
    String name;
    String breed;

    // Behaviour (methods) — these are the "messages" Dog understands
    void bark() {
        System.out.println(name + " says: Woof! Woof!");
    }

    void eat(String food) {
        System.out.println(name + " is eating " + food);
    }

    void introduce() {
        System.out.println("Hi! I am " + name + ", a " + breed);
    }
}

// ■■ Main class ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Main {
    public static void main(String[] args) {

        Dog d1 = new Dog();       // Create object d1
        d1.name  = "Bruno";
        d1.breed = "Labrador";

        // Sending messages to d1
        d1.introduce();           // Message: introduce yourself
        d1.bark();                // Message: please bark
        d1.eat("biscuits");       // Message: eat biscuits

        Dog d2 = new Dog();       // Create a second dog object
        d2.name  = "Milo";
        d2.breed = "Poodle";
        d2.bark();                // Milo gets the same message
    }
```

```
}
```

Output:

```
Hi! I am Bruno, a Labrador
Bruno says: Woof! Woof!
Bruno is eating biscuits
Milo says: Woof! Woof!
```

■ **Remember:** Each object is independent. d1 and d2 are both Dogs but they have different names. Sending bark() to d1 is different from sending it to d2.

■ Example — 2 — Calculator

A Calculator object understands messages like add, subtract, multiply. Notice how each method **returns a value** back to the sender.

```
class Calculator {
    // Method that returns the sum of two numbers
    int add(int a, int b) {
        return a + b;
    }

    int subtract(int a, int b) {
        return a - b;
    }

    int multiply(int a, int b) {
        return a * b;
    }

    // Integer division
    int divide(int a, int b) {
        return a / b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();   // Create the object

        // Send messages and collect replies
        int sum  = calc.add(10, 5);           // Message: add 10 and 5
        int diff = calc.subtract(10, 5);      // Message: subtract 5 from 10
        int prod = calc.multiply(4, 3);       // Message: multiply 4 and 3

        System.out.println("Sum      = " + sum);
```

```
        System.out.println("Diff     = " + diff);
        System.out.println("Product  = " + prod);
    }
}
```

Output:

```
Sum      = 15
Diff     = 5
Product  = 12
```

■ Example — 3 — Bank Account

A BankAccount object protects its balance. You can only change it by sending deposit or withdraw messages. This is called **encapsulation** — an important OOP concept.

```
class BankAccount {
    String owner;
    double balance;

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited Rs." + amount +
                           " | New Balance: Rs." + balance);
    }

    void withdraw(double amount) {
        if (amount > balance) {
            System.out.println("Sorry! Not enough balance.");
        } else {
            balance = balance - amount;
            System.out.println("Withdrawn Rs." + amount +
                               " | New Balance: Rs." + balance);
        }
    }

    void checkBalance() {
        System.out.println(owner + "'s Balance: Rs." + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.owner   = "Aryan";
        acc.balance = 1000.0;

        acc.checkBalance();          // Message: show balance
        acc.deposit(500.0);          // Message: deposit 500
        acc.withdraw(200.0);         // Message: withdraw 200
        acc.withdraw(2000.0);        // Message: withdraw 2000 (should fail)
    }
}
```

Output:

```
Aryan's Balance: Rs.1000.0
Deposited Rs.500.0 | New Balance: Rs.1500.0
Withdrawn Rs.200.0 | New Balance: Rs.1300.0
Sorry! Not enough balance.
```

■ **Tip:** The BankAccount object decides whether the withdrawal is valid. The main method just sends a message — it doesn't check the balance itself. Objects are responsible for their own logic!

■ Example — 4 — Objects Calling Each Other

This is the most powerful idea: **one object can send a message to another object**. Here a Chef object uses a Kitchen object to cook.

```
class Kitchen {
    String stoveStatus = "off";

    void turnOnStove() {
        stoveStatus = "on";
        System.out.println("Kitchen: Stove is now ON.");
    }

    void cook(String dish) {
        if (stoveStatus.equals("on")) {
            System.out.println("Kitchen: Cooking " + dish + "...");
        } else {
            System.out.println("Kitchen: Please turn on the stove first!");
        }
    }

    void turnOffStove() {
        stoveStatus = "off";
        System.out.println("Kitchen: Stove is now OFF.");
    }
}

class Chef {
    String name;
    Kitchen kitchen;          // Chef HAS-A Kitchen (composition)

    void prepareMeal(String dish) {
        System.out.println(name + ": Starting meal preparation.");
        kitchen.turnOnStove();        // Chef sends message to Kitchen
        kitchen.cook(dish);           // Another message to Kitchen
        kitchen.turnOffStove();       // Another message to Kitchen
```

```
        System.out.println(name + ": Meal is ready!");
    }
}

public class Main {
    public static void main(String[] args) {
        Kitchen k = new Kitchen();
        Chef chef  = new Chef();
        chef.name    = "Ramesh";
        chef.kitchen = k;              // Link the objects

        chef.prepareMeal("Biryani");   // Send message to Chef
    }
}
```

Output:

```
Ramesh: Starting meal preparation.
Kitchen: Stove is now ON.
Kitchen: Cooking Biryani...
Kitchen: Stove is now OFF.
Ramesh: Meal is ready!
```

■ **Remember:** When chef.prepareMeal() runs, the Chef object itself sends three messages to the Kitchen object. This chain of message passing IS computation!

■ Example — 5 — Remote Control & TV

A RemoteControl object sends messages to a TV object. The remote doesn't know the inner workings of the TV — it just presses buttons (sends messages).

```
class TV {
    boolean isOn   = false;
    int     volume = 10;
    int     channel = 1;

    void powerToggle() {
        isOn = !isOn;
        System.out.println("TV: Power " + (isOn ? "ON" : "OFF"));
    }

    void increaseVolume() {
        if (isOn) { volume++; System.out.println("TV Volume: " + volume); }
    }

    void changeChannel(int ch) {
        if (isOn) { channel = ch; System.out.println("TV Channel: " + ch); }
    }
}

class RemoteControl {
    TV tv;                          // Remote controls a TV

    void pressPower()         { tv.powerToggle(); }
    void pressVolumePlus()    { tv.increaseVolume(); }
    void pressChannel(int ch) { tv.changeChannel(ch); }
}

public class Main {
    public static void main(String[] args) {
        TV myTV          = new TV();
        RemoteControl rc = new RemoteControl();
        rc.tv = myTV;

        rc.pressPower();           // Turn TV on
        rc.pressVolumePlus();      // Volume up
        rc.pressVolumePlus();      // Volume up again
        rc.pressChannel(5);        // Go to channel 5
        rc.pressPower();           // Turn TV off
    }
}
```

Output:

```
TV: Power ON
TV Volume: 11
TV Volume: 12
TV Channel: 5
TV: Power OFF
```

■ Example — 6 — Student & Report Card

A Student object asks a ReportCard object to calculate and display results. Each class has a clear responsibility.

```
class ReportCard {
    String studentName;
    int maths, science, english;

    int totalMarks() {
        return maths + science + english;
    }

    double percentage() {
        return (totalMarks() / 300.0) * 100;
    }

    String grade() {
        double p = percentage();
        if      (p >= 90) return "A+";
        else if (p >= 75) return "A";
        else if (p >= 60) return "B";
        else if (p >= 40) return "C";
        else              return "D";
    }

    void printReport() {
        System.out.println("========== Report Card ==========");
        System.out.println("Name    : " + studentName);
        System.out.println("Maths   : " + maths);
        System.out.println("Science : " + science);
        System.out.println("English : " + english);
        System.out.println("Total   : " + totalMarks() + " / 300");
        System.out.println("Percent : " + percentage() + "%");
        System.out.println("Grade   : " + grade());
        System.out.println("================================");
```

```
    }
}

public class Main {
    public static void main(String[] args) {
        ReportCard rc = new ReportCard();
        rc.studentName = "Priya";
        rc.maths       = 88;
        rc.science     = 92;
        rc.english     = 79;

        rc.printReport();                    // Send one message — does everything

        // We can also ask individual questions (send individual messages)
        System.out.println("Grade only: " + rc.grade());
    }
}
```

Output:

```
========== Report Card ==========
Name    : Priya
Maths   : 88
Science : 92
English : 79
Total   : 259 / 300
Percent : 86.33333333333333%
Grade   : A
================================
Grade only: A
```

■ Example — 7 — Alarm Clock

This example shows a method calling another method *within* the same object — even that is message passing!

```
class AlarmClock {
    int  hours   = 6;
    int  minutes = 0;
    boolean alarmSet = false;

    void setTime(int h, int m) {
        hours   = h;
        minutes = m;
        System.out.println("Clock set to " + hours + ":" + formatMin(minutes));
    }

    String formatMin(int m) {        // helper — called by other methods
        return (m < 10) ? "0" + m : "" + m;
    }

    void setAlarm() {
        alarmSet = true;
        System.out.println("Alarm ON for " + hours + ":" + formatMin(minutes));
    }

    void ring() {
        if (alarmSet) {
            System.out.println("RING RING! Time is " + hours + ":" + formatMin(minutes))
;
            alarmSet = false;        // reset after ringing
        } else {
            System.out.println("No alarm was set.");
        }
    }

    void snooze(int extraMinutes) {
        minutes = minutes + extraMinutes;
        if (minutes >= 60) {
            hours   = hours + 1;
            minutes = minutes - 60;
        }
        setAlarm();                  // calling another method on SAME object
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        AlarmClock ac = new AlarmClock();

        ac.setTime(7, 45);          // Message: set time
        ac.setAlarm();              // Message: set alarm
        ac.ring();                  // Message: ring now
        ac.setTime(8, 0);
        ac.snooze(10);              // Message: snooze for 10 minutes
        ac.ring();                  // Ring at new snoozed time
    }
}
```

Output:

```
Clock set to 7:45
Alarm ON for 7:45
RING RING! Time is 7:45
Clock set to 8:00
Alarm ON for 8:10
RING RING! Time is 8:10
```

■ **Tip:** Inside snooze(), the line setAlarm() is a message sent from the AlarmClock object to itself. Every method call — even internal ones — is message passing.

11. Return Values from Methods

When you send a message, the object can **reply with data**. The reply is called a **return value**. Use **void** when there is no reply, or the data type (int, double, String…) when the method sends data back.

Return Type	Meaning	Example Method
void	No reply needed	void bark() { ... }
int	Reply is a whole number	int add(int a, int b) { return a+b; }
double	Reply is a decimal	double percentage() { return ...; }
String	Reply is text	String grade() { return "A"; }
boolean	Reply is true/false	boolean isAdult() { return age>=18; }

12. Parameters — Adding Details to a Message

Sometimes when you send a message, you need to include extra information. In Java, this extra information is passed as **parameters** (also called arguments).

```
class Printer {
    void printMessage(String msg) {          // one parameter
        System.out.println(msg);
    }

    void printRepeat(String msg, int times) { // two parameters
        for (int i = 0; i < times; i++) {
            System.out.println(msg);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Printer p = new Printer();

        p.printMessage("Hello Class 9!");           // 1 argument
        p.printRepeat("Study hard!", 3);            // 2 arguments
    }
}
```

Output:

```
Hello Class 9!
Study hard!
Study hard!
Study hard!
```

13. Quick Revision Table

Term	Meaning	Real-life Analogy
Class	Blueprint / template for objects	Design of a house
Object	A real instance created from class	An actual house built
Field / Attribute	Data stored in the object	Colour, size of house
Method	Action / behaviour of an object	Opening the door
Message Passing	Calling a method on an object	Knocking on the door
Parameter	Extra info sent with the message	Knocking 3 times
Return Value	Data sent back as the reply	Owner opens & replies

void	Method sends no reply back	One-way instruction
new keyword	Creates a new object	Building a new house
Dot operator (.)	Access method or field of object	Ringing a specific bell

Key Takeaways

✔ Every Java program is a collection of **objects** that talk to each other.

✔ Objects talk by **calling methods** — this is called **message passing**.

✔ The **dot operator (.)** is how you send a message: objectName.methodName()

✔ Methods can **accept parameters** (extra info) and **return values** (replies).

✔ One object can hold a **reference to another object** and call its methods.

✔ Each object is responsible for **its own data and logic** — this keeps code clean.

Practice Tip: Try creating your own classes — a Library, a MusicPlayer, or a GameCharacter — give them fields and methods, create objects, and send messages!